

Source code:

```
void main() {  
  
    TRISB=0x00 ; // set portb as output  
  
    portb=0x00 ; //initialize portb as off  
  
    while (1)  
    {  
  
        portb.f0=1;  
  
        delay_ms(1000);  
  
        portb.f0=0;  
  
        delay_ms(1000);  
  
    }  
}
```

Output:

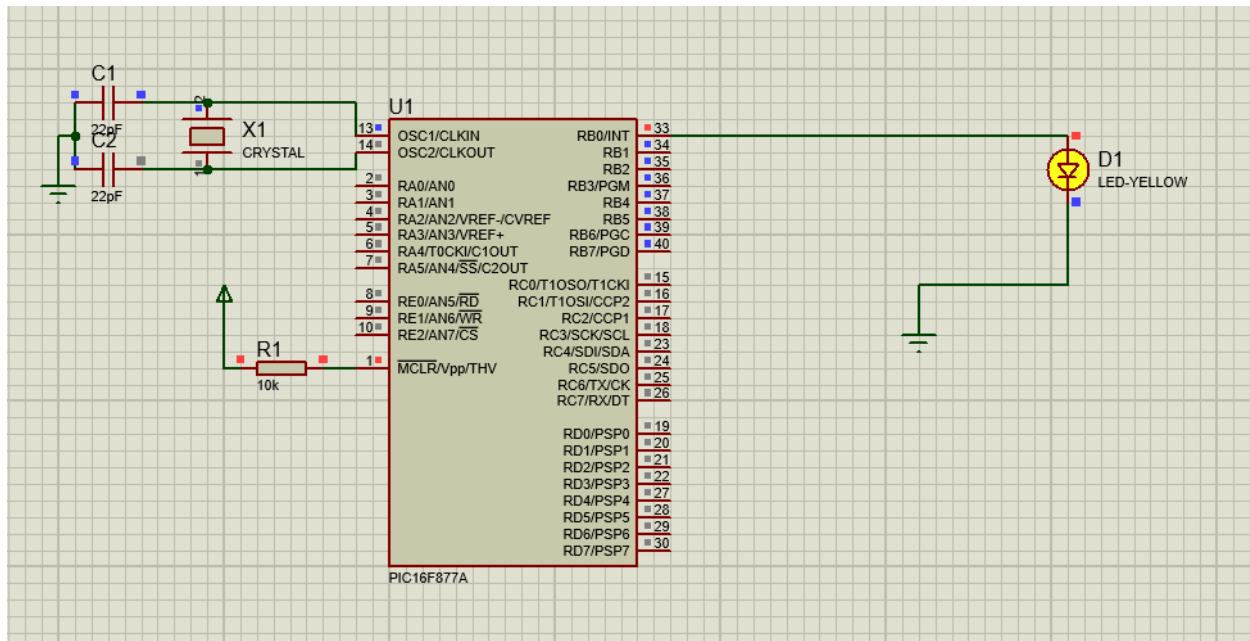


Figure-1: LED flasher using PIC microcontroller.

Source code:

```
char arr[] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

int i;

void main() {

    TRISB = 0x00;

    PORTB = 0x00;

    while(1){

        for(i = 0; i < 10; i++){

            PORTB = arr[i];

            Delay_ms(100);

        }

    }
```

Output:

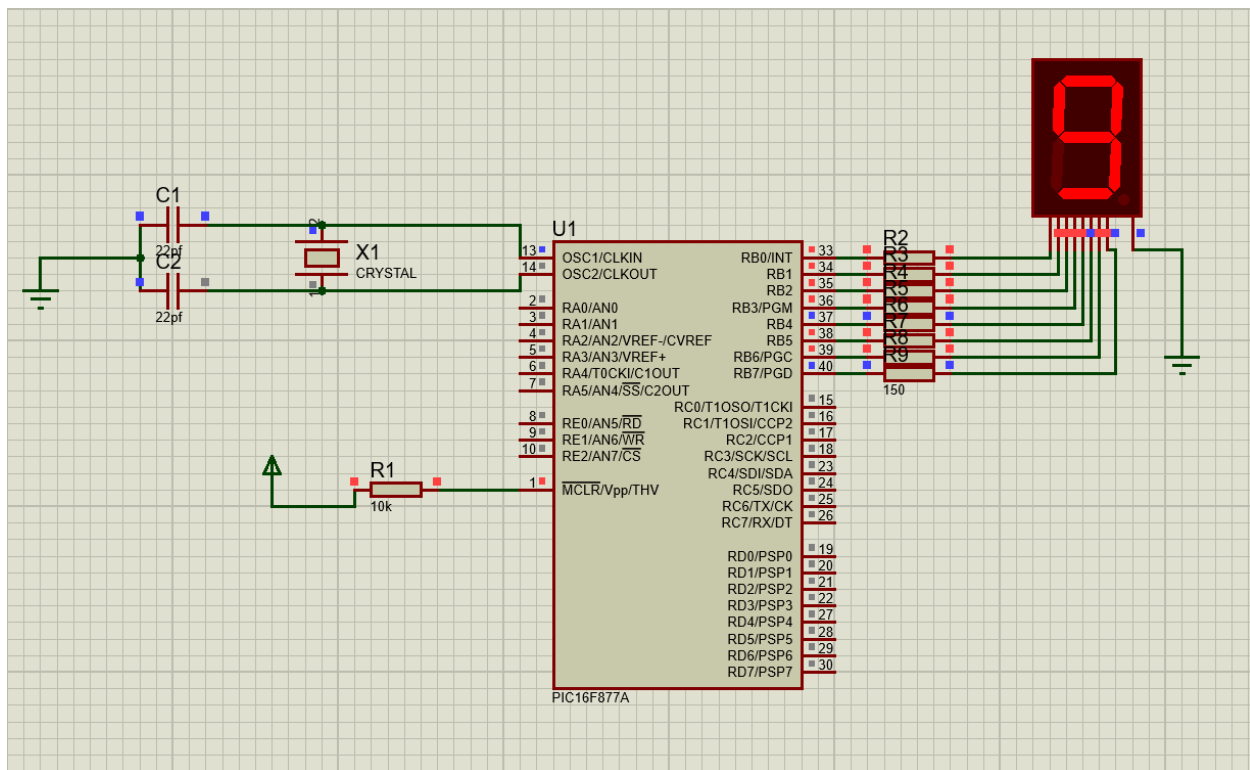


Figure-2: count 0 to 9 in 7 segment display.

Source code:

```
char arr[] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

int i;

void main() {

    TRISB = 0x00;

    PORTB = 0x00;

    while(1){

        for(i = 0; i < 10; i++){

            PORTB = arr[i];

            Delay_ms(100);

        }

    }

}
```

Output:

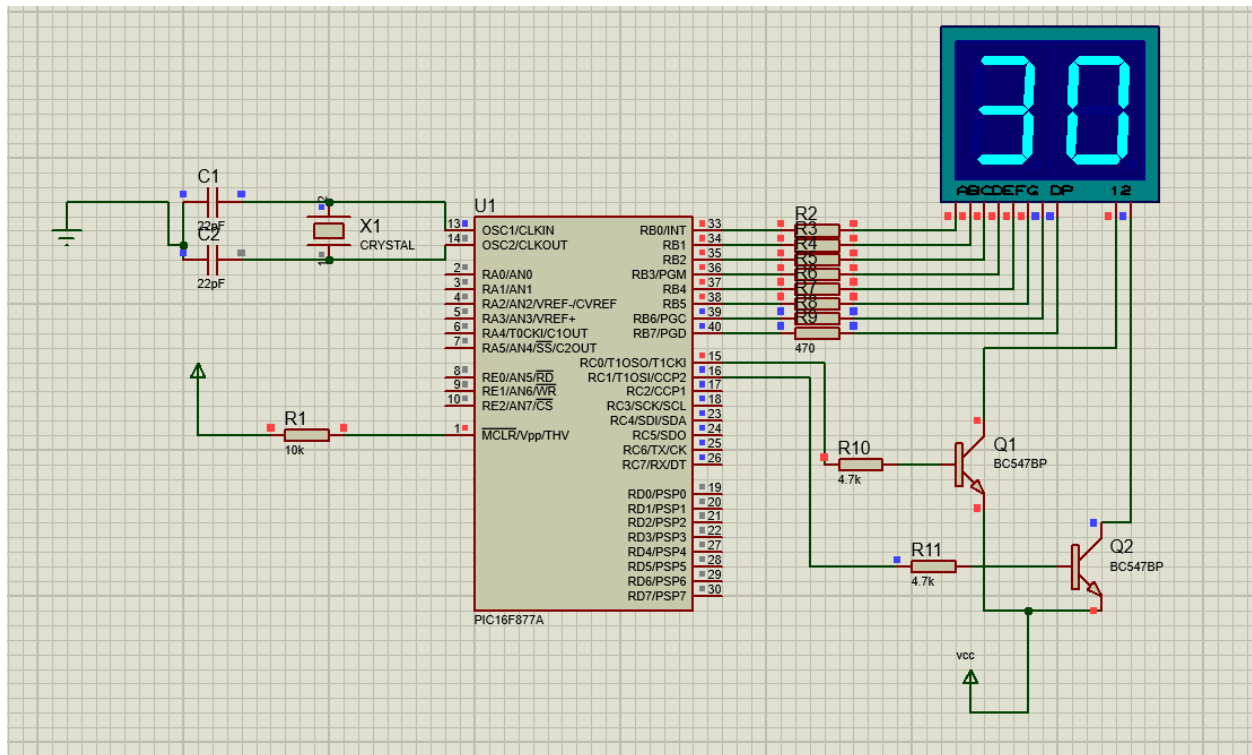


Figure-3:7-segment multiplexer.

Source code:

```
void main() {  
    TRISB = 0x00;  
    PORTB = 0x00;  
    while(1){  
        PORTB.f2=1; //en1 pin active  
  
        PORTB.f0 = 1;  
  
        PORTB.f1 = 0;  
  
        delay_ms(5000);  
  
        PORTB.f0 = 0;  
  
        PORTB.f1 = 1;  
  
        delay_ms(5000);  
    }  
}
```

Output:

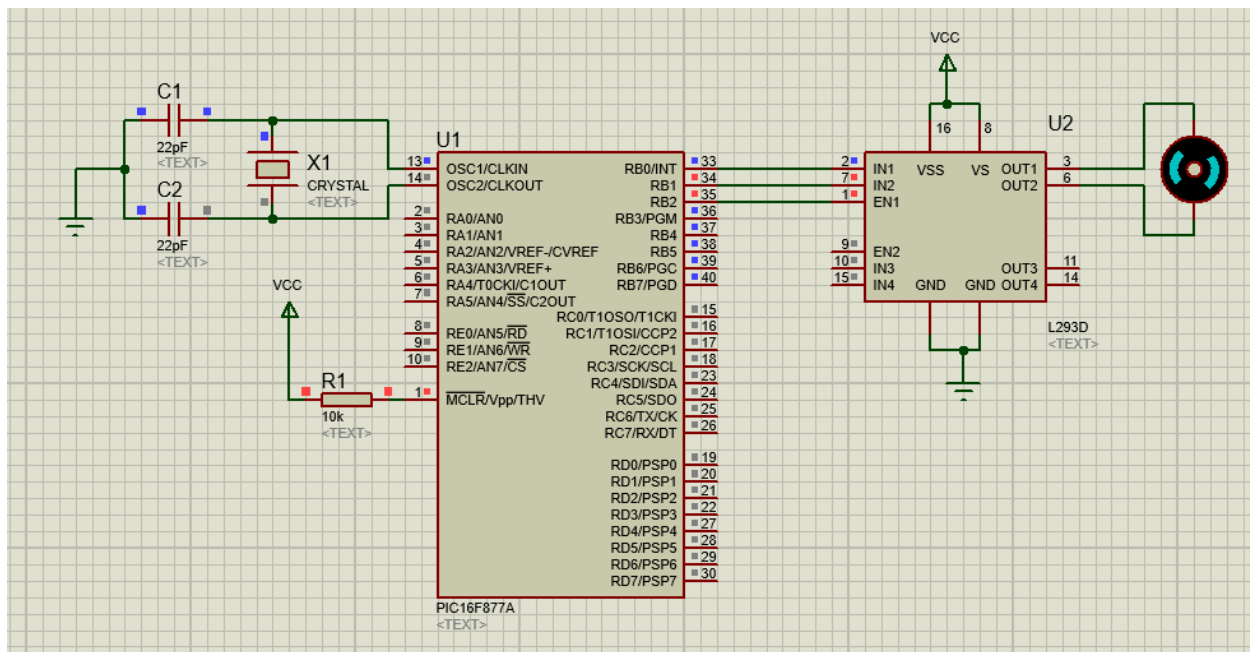


Figure-6: DC motor speed using PIC microcontroller

Source code:

```
int i;

void main() {

    TRISB.F0 = 0; // RB0 as output

    PORTB.F0 = 0;

    while(1){

        // LEFT 90 degree

        for(i=0; i<50; i++){

            PORTB.F0 = 1;

            Delay_us(1000); // 1 ms pulse

            PORTB.F0 = 0;

            Delay_us(19000); // remainder of 20 ms }

        Delay_ms(500);

        // STOP

        for(i=0; i<50; i++){

            PORTB.F0 = 1;

            Delay_us(1500); // 1.5 ms pulse

            PORTB.F0 = 0;

            Delay_us(18500); }

        Delay_ms(500);

        // RIGHT 90 degree

        for(i=0; i<50; i++){

            PORTB.F0 = 1;

            Delay_us(2000); // 2 ms pulse

            PORTB.F0 = 0;

            Delay_us(18000); }

        Delay_ms(500);

        // STOP

        for(i=0; i<50; i++){
```

```

PORTB.F0 = 1;

Delay_us(1500); // 1.5 ms pulse

PORTB.F0 = 0;

Delay_us(18500);

}

Delay_ms(500);

}

}

```

Output:

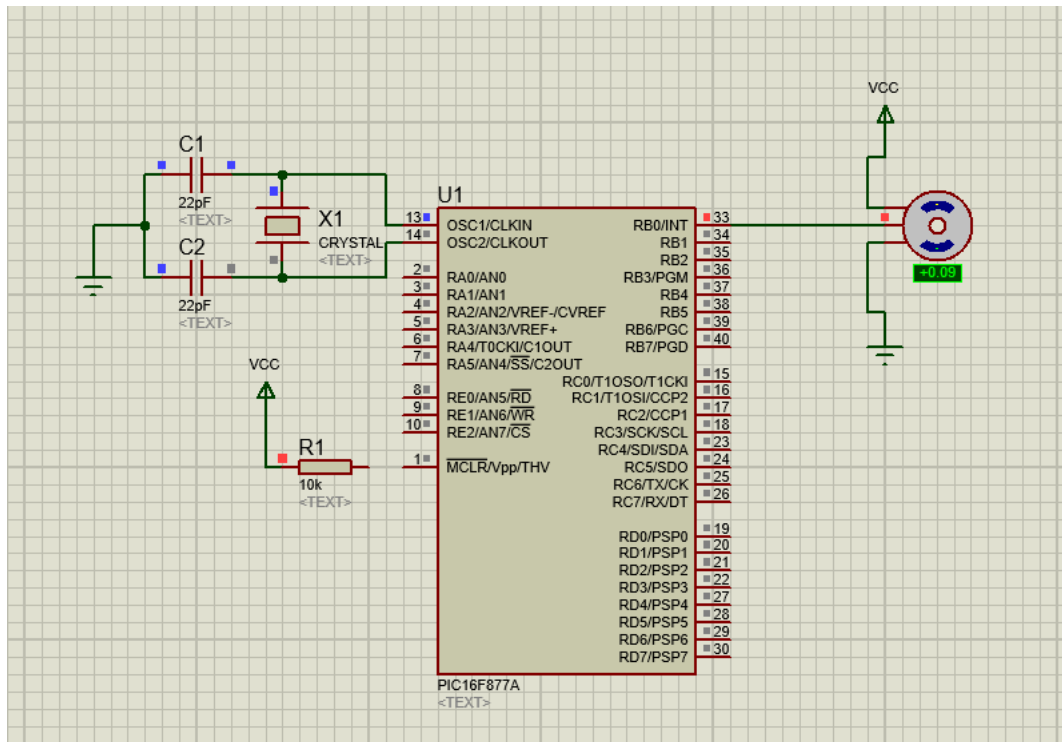


Figure-7: Servo motor using PIC microcontroller

Source code:

```
// LCD module connections

sbit LCD_RS at RB4_bit;

sbit LCD_EN at RB5_bit;

sbit LCD_D4 at RB0_bit;

sbit LCD_D5 at RB1_bit;

sbit LCD_D6 at RB2_bit;

sbit LCD_D7 at RB3_bit;

sbit LCD_RS_Direction at TRISB4_bit;

sbit LCD_EN_Direction at TRISB5_bit;

sbit LCD_D4_Direction at TRISB0_bit;

sbit LCD_D5_Direction at TRISB1_bit;

sbit LCD_D6_Direction at TRISB2_bit;

sbit LCD_D7_Direction at TRISB3_bit;

int temp;

char temper[7];

void READ_temp(void)
{
    temp = ADC_Read(0);
    temp = temp * 5/1023;
    temp = temp * 100;}

void data_conversion(void)
{ inttostr(temp, temper);}

void display_temperature(void)
{ lcd_out(1, 1, "TEMPERATURE=");

  lcd_out(1, 13, Ltrim(temper));

  Lcd_Chrcp(0xdf);

  Lcd_Chrcp('C');

  Lcd_Chrcp(' ');

}
```

```

void main()
{
    ADC_Init();

    Lcd_Init(); // Initialize LCD

    Lcd_Cmd(_LCD_CLEAR); // Clear display

    lcd_cmd(_LCD_CURSOR_OFF);

    lcd_out(1, 4, "DIGITAL TEMPERATURE");

    lcd_out(2, 6, "SENSOR");

    delay_ms(1000);

    Lcd_Cmd(_LCD_CLEAR); // Clear display

    while(1)
    {
        READ_temp();

        data_conversion();

        display_temperature();

    }
}

```

Output:

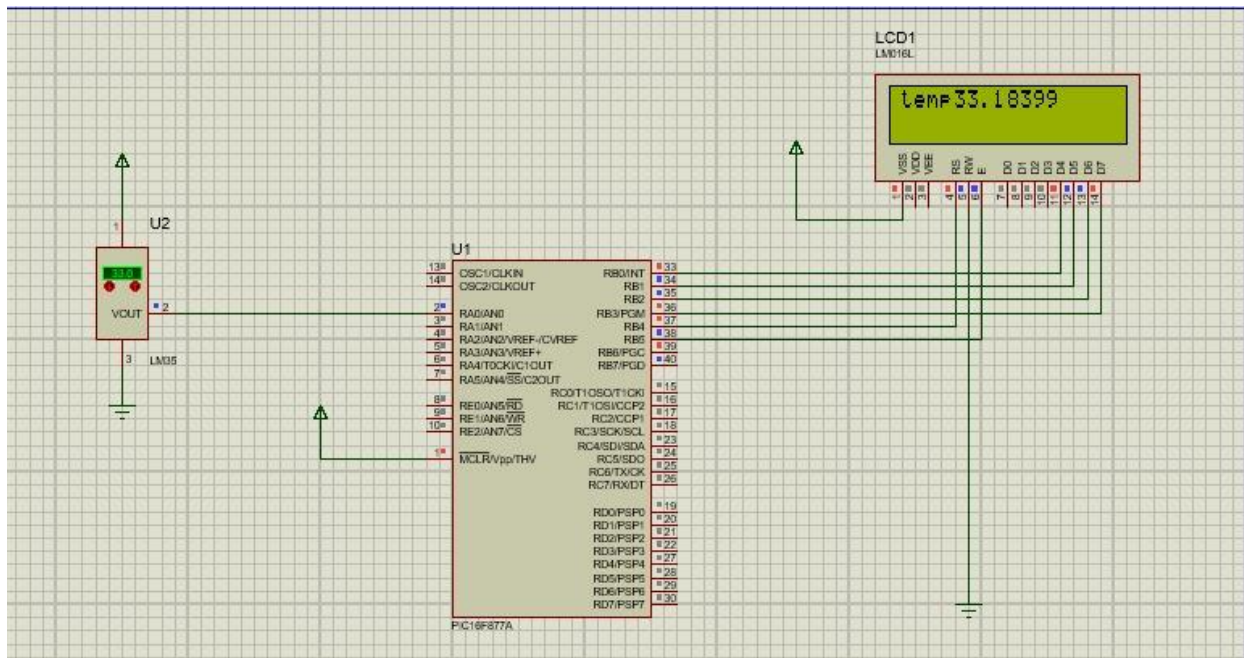


Figure-10: sensor input from LM35 temperature sensor and display corresponding sensor value through LCD display.

Source code:

```
void main(){  
  
TRISD = 0b00000000;  
  
PORTD = 0b11111111;  
  
do{  
  
PORTD = 0b00000011;  
  
Delay_ms(500);  
  
PORTD = 0b00000110;  
  
Delay_ms(500);  
  
PORTD = 0b00001100;  
  
Delay_ms(500);  
  
PORTD = 0b00001001;  
  
Delay_ms(500);  
  
}while(1);}
```

Output:

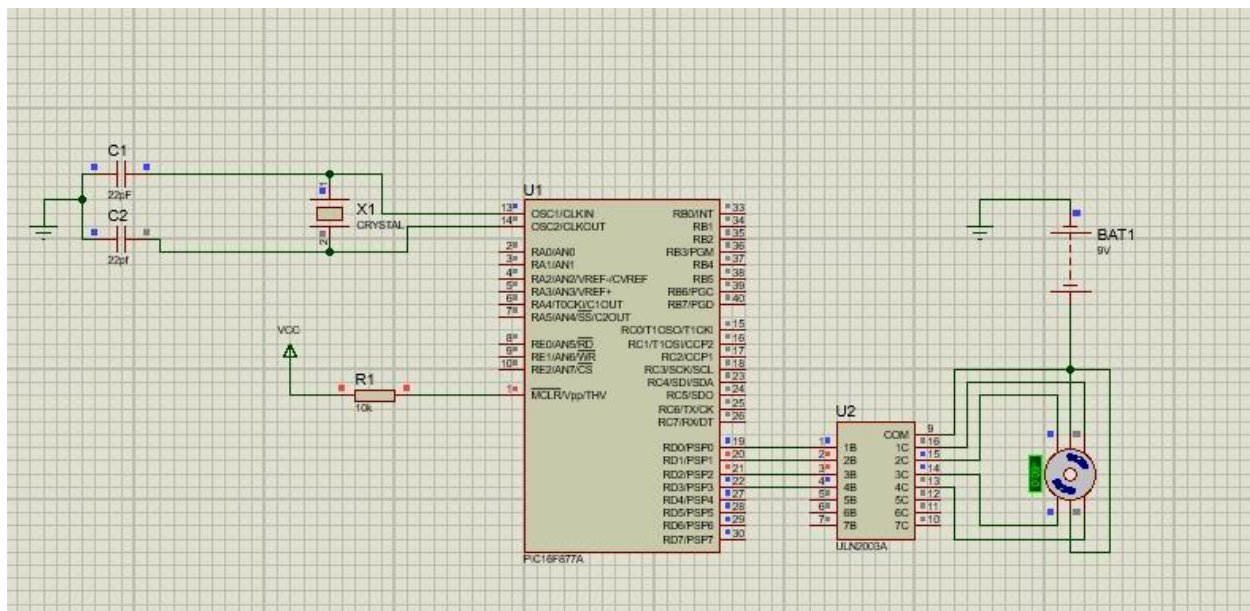


Figure-11: Stepper motor using PIC microcontroller.

```
unsigned int adc_value;

char txt[7];

void main() {  ADCON1 = 0x8E;    // AN0 analog

    TRISA = 0xFF;    // PORTA input (AN0 input)

    TRISC = 0x80;    // RC7 input, RC6 output

    UART1_Init(9600);  // UART 9600 baud

    Delay_ms(1000);

    while(1) {

        adc_value = ADC_Read(0);

        IntToStr(adc_value, txt);

        UART1_Write_Text("ADC Value = ");

        UART1_Write_Text(txt);

        UART1_Write_Text("\r\n");

        Delay_ms(1000);  }}

```

The screenshot displays a Proteus simulation of a PIC16F877A microcontroller circuit. The circuit includes a 5V regulator (RV1), a crystal oscillator (X1), and a 10k pull-up resistor (R1). The PIC16F877A is connected to a serial terminal (U1) which displays the output of the ADC module. The serial terminal shows a repeating sequence of 'ADC Value = 164'. The PIC16F877A pinout is visible on the right side of the component.

Figure-8: Analog to Digital (ADC) conversion feature of PIC microcontroller.

Source code:

```
sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB4_bit;
sbit LCD_D5 at RB5_bit;
sbit LCD_D6 at RB6_bit;
sbit LCD_D7 at RB7_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB4_bit;
sbit LCD_D5_Direction at TRISB5_bit;
sbit LCD_D6_Direction at TRISB6_bit;
sbit LCD_D7_Direction at TRISB7_bit;
// End LCD module connections
char txt1[] = "mikroElektronika";
char txt2[] = "EasyPIC6";
char txt3[] = "Lcd4bit";
char txt4[] = "example";
char i;                // Loop variable
void Move_Delay() {
    Delay_ms(500);}
void main(){
    TRISA = 0xFF;        // Set PORTA as inputs
    TRISB = 0x00;        // Set PORTB as outputs
    PORTB = 0x00;        // Clear PORTB
    // Important: Power-up delay for LCD
    Delay_ms(200);
    // Configure ADC and comparators
    ADCON1 = 0x06;        // Set all PORTB pins as digital I/O
    CMCON = 0x07;        // Disable comparators (if available on your PIC)
    // Initialize LCD
    Lcd_Init();           // Initialize LCD
    Delay_ms(50);         // Extra delay after init
    // Clear display and turn cursor off
    Lcd_Cmd(_LCD_CLEAR);  // Clear display
    Lcd_Cmd(_LCD_CURSOR_OFF); // Cursor off
    // Display initial static text
    Lcd_Out(1,1,txt1);     // Write text in first row
    Lcd_Out(2,1,txt2);     // Write text in second row
    Delay_ms(3000);        // Show static text for 3 seconds
    // Clear display for moving text demo
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1,6,txt3);     // Write text in first row
```

```

Lcd_Out(2,6,txt4);          // Write text in second row
Delay_ms(2000);
Lcd_Cmd(_LCD_CLEAR);
Lcd_Out(1,1,txt1);
Lcd_Out(2,5,txt2);
Delay_ms(2000);
// Moving text to the right
for(i=0; i<4; i++) {
    Lcd_Cmd(_LCD_SHIFT_RIGHT);
    Move_Delay(); }
// Continuous moving text
while(1) {                  // Endless loop
    for(i=0; i<12; i++) {    // Move text to the left 12 times
        Lcd_Cmd(_LCD_SHIFT_LEFT);
        Move_Delay(); }
    for(i=0; i<12; i++) {     // Move text to the right 12 times
        Lcd_Cmd(_LCD_SHIFT_RIGHT);
        Move_Delay();}}

```

Output:

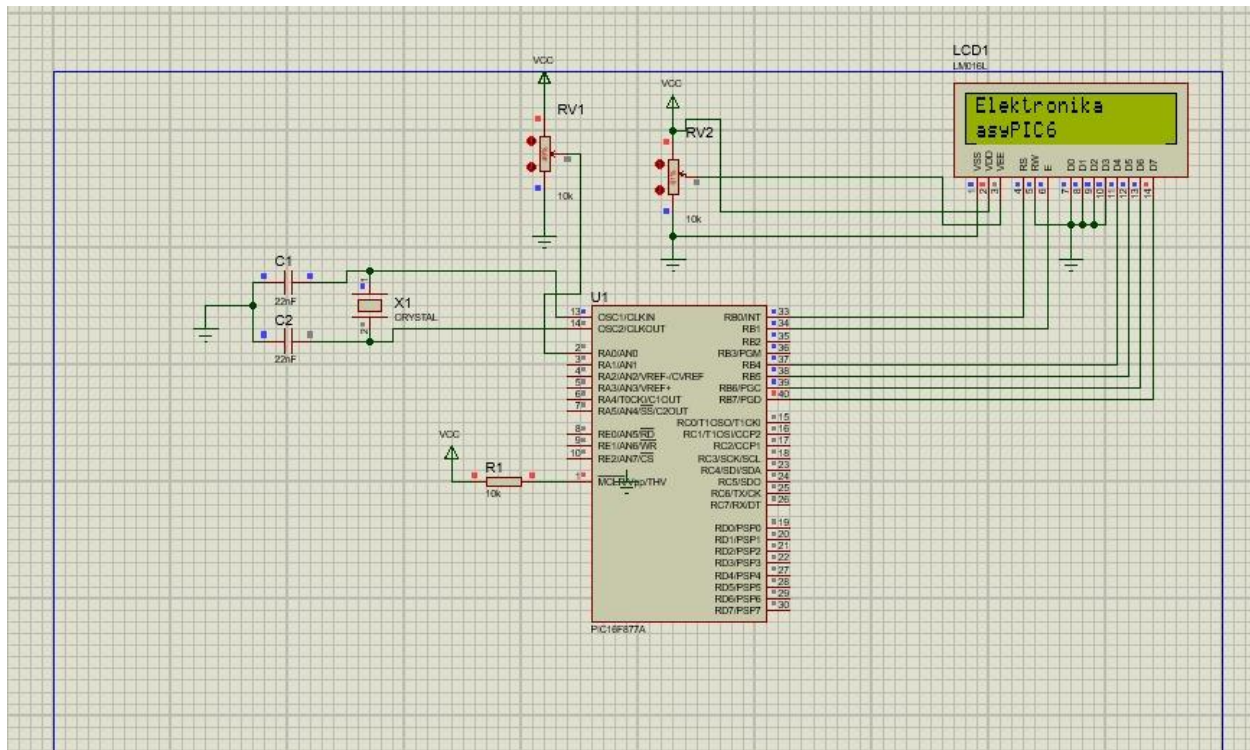


Figure-9: Interface and Display Data on LCD using PIC Microcontroller.

Source code:

```
unsigned char count;

unsigned char seg[] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

void main(){

    ADCON1 = 0x06;

    TRISB = 0x00;

    count = EEPROM_Read(0x00);

    if(count > 9) count = 0;

    while(1){

        PORTB = seg[count];

        Delay_ms(1000);

        count++;

        if(count > 9) count = 0;

        EEPROM_Write(0x00, count);

        Delay_ms(20);}}
```

Output :

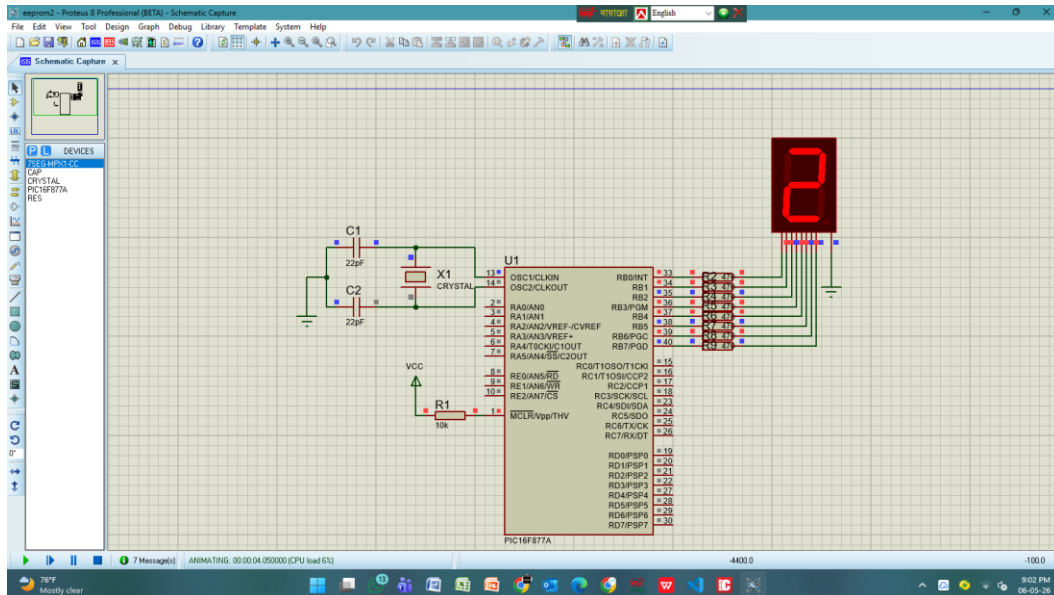


Figure-12: Interface EEPROM for Data Storage and Retrieval using PIC Microcontroller

```
void main() {  
    TRISB = 0x00; // PORTB output  
    PORTB = 0x00;  
  
    while(1) {  
        PORTB.F0 = 1; // Relay ON  
        delay_ms(1000);  
  
        PORTB.F0 = 0; // Relay OFF  
        delay_ms(1000);  
    }  
}
```

Figure-5: control a high voltage load using mechanical relay.

Source code:

```
char arr[] = {0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x10};

void main() {

    int count=0,left,right;

    TRISB = TRISC = 0x00;

    TRISD = 0xff;

    PORTB = PORTC = PORTD = 0x00;

    while(1){

        if(PORTD.F0 == 1){

            Delay_ms(30);    // debounce

            if(PORTD.F0 == 1){

                count++;

                if(count == 100) count = 0;

                while(PORTD.F0 == 1); }// wait till release  }

                //decrease button

            if(PORTD.F1 == 1){

                Delay_ms(30);    // debounce

                if(PORTD.F1 == 1){

                    if(count == 0) count = 0;

                    else count--;

                    while(PORTD.F1 == 1); }// wait till release

                    //reset button

                if(PORTD.F2 == 1){

                    delay_ms(30);

                    if(PORTD.F2 == 1) {

                        count = 0; }}

                    left = count/10;

                    right = count%10;

                    PORTC.F0 = 1;
```

```
PORTB = arr[left]; //segment show
```

```
Delay_ms(5);
```

```
PORTC.F0 = 0;
```

```
PORTC.F1 = 1;
```

```
PORTB = arr[right];
```

```
Delay_ms(5);
```

```
PORTC.F1 = 0; }}
```

Output:

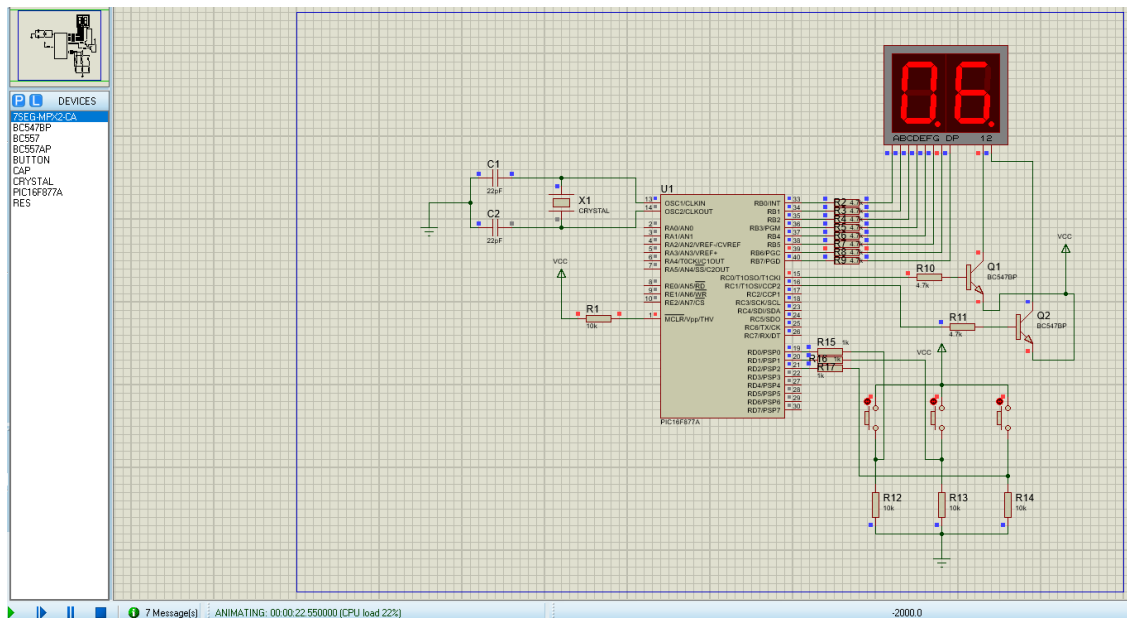


Figure-4: interface double digit 7 segment LED display using push button.

Source code:

```
void MSDelay(unsigned char Time){
    unsigned char y,z;
    for(y=0x00; y<Time; y++)
        for(z=0x00; z<0x14; z++);}

void main(){
    TRISC = 0x00;
    TRISD = 0x00;
    while(0x01) {
        PORTD = 0x80;
        PORTC = 0x00;
        MSDelay(0x0A);
        PORTD = 0x40;
        PORTC = 0xFF;
        MSDelay(0x0A);
        PORTD = 0x20;
        PORTC = 0xFF;
        MSDelay(0x0A);
        PORTD = 0x10;
        PORTC = 0xD8;
        MSDelay(0x0A);
        PORTD = 0x08;
        PORTC = 0xD8;
        MSDelay(0x0A);
        PORTD = 0x04;
        PORTC = 0xFF;
        MSDelay(0x0A);
        PORTD = 0x02;
        PORTC = 0xFF;
        MSDelay(0x0A);
        PORTD = 0x01;
        PORTC = 0x00;
        MSDelay(0x0A); } }
```

Output:

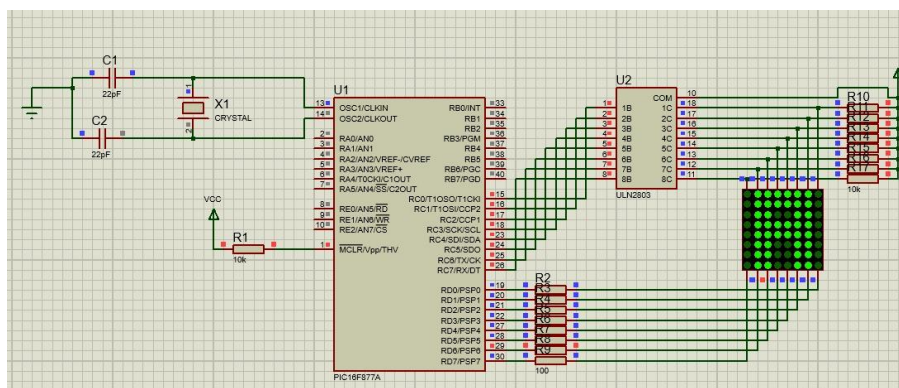


Figure-13: display character through dot matrix display.